

PulseDesk

Analytics Service

Documentação Técnica

Arquitetura baseada em Microserviços e Eventos

2026



Sumário

1	Visão Geral	2
1.1	Responsabilidade	2
1.2	Papel na arquitetura	2
1.3	Principais responsabilidades	3
2	Estrutura do Microserviço	3
3	Fluxo Principal	4
4	Análise dos Arquivos	5
4.1	AnalyticsServiceApplication.java	5
4.2	HistoricalMarketDataEventConsumer.java	6
4.3	AnalyticsService.java	6
4.4	AnalyticsEventProducer.java	6
5	Integrações Externas	7
6	Comunicação por Eventos	7
6.1	Evento consumido	7
6.2	Evento produzido	8
6.3	market-data.updated	8
6.4	Motivação	9
7	Contratos de Eventos	9
7.1	HistoricalMarketDataUpdated	9
7.2	AnalyticsUpdated	10
7.3	Schema Registry	10
8	Configuração	11
8.1	application.yml	11
9	Dependências	11
9.1	pom.xml	11
10	Docker	12

1 Visão Geral

1.1 Responsabilidade

O Analytics Service é o microserviço responsável por transformar dados de mercado recebidos através do Kafka em informações analíticas utilizadas pelo PulseDesk.

Na implementação atual, o serviço recebe séries históricas de preços de fechamento produzidas pelo Market Data Service e utiliza esses valores para calcular a volatilidade do ativo. O resultado do processamento é representado por um novo evento analítico e publicado novamente no Kafka.

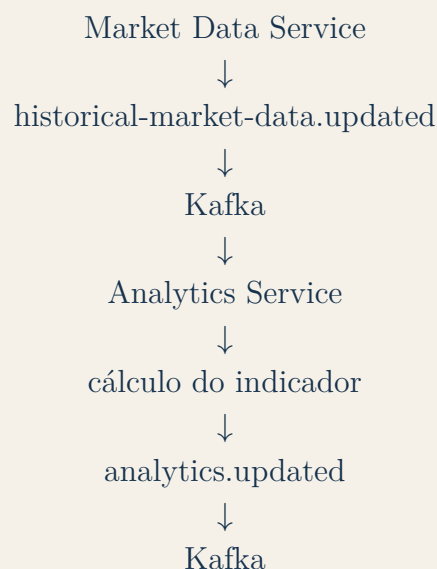
Dessa forma, a obtenção dos dados de mercado permanece responsabilidade do Market Data Service, enquanto o Analytics Service concentra a lógica necessária para transformar esses dados em indicadores derivados.

1.2 Papel na arquitetura

O Analytics Service ocupa uma posição intermediária no fluxo orientado a eventos do PulseDesk.

Ele não consulta diretamente APIs externas e também não recebe requisições HTTP do frontend durante o fluxo atual. Sua entrada é formada por eventos Kafka e sua saída também ocorre através de eventos.

O fluxo principal pode ser representado da seguinte forma:



Essa organização mantém os microserviços desacoplados. O Market Data Service não precisa chamar diretamente o Analytics Service. Ele apenas publica os dados históricos, enquanto o Analytics Service decide como reagir quando recebe esse evento.

1.3 Principais responsabilidades

- consumir eventos de dados históricos de mercado através do Kafka;
- receber séries de preços de fechamento dos ativos;
- validar se existem dados suficientes para realizar o cálculo;
- selecionar os 21 preços mais recentes necessários para a análise;
- calcular 20 retornos percentuais consecutivos;
- calcular a volatilidade a partir desses retornos;
- criar um evento AnalyticsUpdated contendo o resultado da análise;
- publicar o resultado no tópico analytics.updated;
- manter a relação entre o evento analítico produzido e o evento histórico que originou seu cálculo.

Na implementação atual, o indicador efetivamente calculado pelo Analytics Service é VOLATILITY20D.

O tópico market-data.updated faz parte da arquitetura geral do PulseDesk, porém não é consumido pelo Analytics Service na versão atual apresentada nesta documentação. Da mesma forma, o indicador PERCENTAGECHANGE não é calculado pela implementação atual do AnalyticsService.

2 Estrutura do Microserviço

O Analytics Service possui uma estrutura reduzida quando comparado a outros microserviços do PulseDesk, pois sua responsabilidade está concentrada no processamento de eventos e no cálculo de indicadores.

```
analytics-service/  
|  
|-- consumer/  
|   '-- HistoricalMarketDataEventConsumer.java  
|-- producer/  
|   '-- AnalyticsEventProducer.java  
|  
|-- service/  
|   '-- AnalyticsService.java  
|  
'-- AnalyticsServiceApplication.java
```



O pacote consumer contém os componentes responsáveis por receber eventos publicados no Kafka. Na implementação atual, `HistoricalMarketDataEventConsumer` escuta eventos contendo dados históricos de mercado.

O pacote service contém a lógica de negócio do microserviço. `AnalyticsService` recebe os preços históricos, realiza as validações necessárias e calcula a volatilidade do ativo.

O pacote producer contém o componente responsável pela saída assíncrona do serviço. `AnalyticsEventProducer` publica o resultado das análises no Kafka.

Por fim, `AnalyticsServiceApplication` é a classe principal da aplicação e inicia o contexto Spring Boot.

Diferentemente do Market Data Service, o Analytics Service não possui Controller, DTOs próprios ou providers externos na implementação atual. Sua comunicação ocorre essencialmente através de eventos Kafka e dos contratos Avro compartilhados pelo PulseDesk.

3 Fluxo Principal

O fluxo do Analytics Service começa quando o Market Data Service publica um evento contendo uma série histórica de preços de fechamento no tópico `historical-market-data.updated`.



O `HistoricalMarketDataEventConsumer` permanece associado ao tópico `historical-market-data.updated` através de um `KafkaListener`.

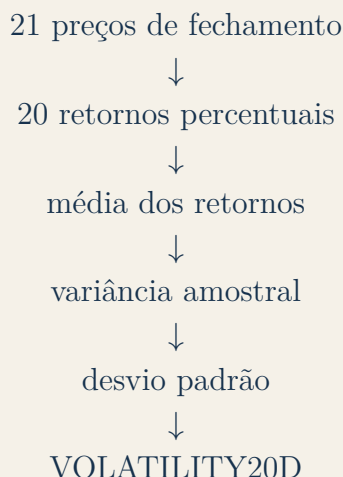
Quando uma nova mensagem é disponibilizada, a infraestrutura do Spring Kafka realiza seu consumo e entrega ao método `consume` um objeto `HistoricalMarketDataUpdated` já desserializado.

O evento é então encaminhado para o `AnalyticsService` através do método `analyze`.

O `AnalyticsService` extrai o símbolo e os preços de fechamento recebidos. Caso existam menos de 21 preços, o processamento é interrompido, pois não seria possível obter os 20 retornos necessários para o indicador atual.

Quando existem dados suficientes, são selecionados os 21 preços mais recentes.

A partir deles, o serviço calcula:



O resultado é utilizado para construir um `AnalyticsUpdated`. Esse evento mantém uma referência ao evento histórico que originou o cálculo e recebe um novo identificador próprio.

Depois que o processamento termina, o `AnalyticsEventProducer` publica o `AnalyticsUpdated` no tópico `analytics.updated`.

O fluxo do serviço é, portanto, completamente orientado a eventos: um evento de dados de mercado provoca um processamento e resulta na criação de um novo evento analítico.

4 Análise dos Arquivos

4.1 `AnalyticsServiceApplication.java`

`AnalyticsServiceApplication` é a classe de entrada do microserviço. Sem esse arquivo, não seria possível inicializar normalmente o Spring Boot dessa aplicação.

O `@SpringBootApplication` inicializa a configuração do Spring Boot, incluindo component scanning e autoconfiguração.

O `main` chama `SpringApplication.run`, que cria o contexto Spring e inicializa os beans (objetos gerenciados pelo Spring) da aplicação.

O `ApplicationContext` acaba sendo um ambiente que o Spring cria para administrar os objetos da aplicação. Isso é a base para a Inversion of Control - em vez de o código controlar diretamente a criação e o ciclo de vida dos objetos, o Spring faz isso - e para Dependency Injection - mecanismo pelo qual o Spring fornece a cada objeto as dependências necessárias para ele funcionar.

4.2 HistoricalMarketDataEventConsumer.java

HistoricalMarketDataEventConsumer escuta o tópico `historical-market-data.updated` do Kafka. Depois disso, ele repassa o evento recebido para o `AnalyticsService` que desenvolve um novo evento com os dados.

A partir disso, ocorre a publicação da análise dos dados dentro do Consumer mesmo. Isso não seria adequado em um sistema maior, já que está inserindo dentro do Consumer uma chamada ao Producer de eventos para o Kafka.

4.3 AnalyticsService.java

O `AnalyticsService` é a camada responsável pelo processamento dos dados históricos de mercado e pelo cálculo dos indicadores analíticos do PulseDesk. O serviço recebe um evento `HistoricalMarketDataUpdated`, extrai o símbolo e os preços de fechamento do ativo, valida se existem dados suficientes para o cálculo e produz como resultado um novo evento `AnalyticsUpdated`.

O método `analyze` recebe o evento contendo os dados históricos do ativo. Primeiro, ele obtém o símbolo e a lista de preços de fechamento. Em seguida, verifica se existem pelo menos 21 preços disponíveis. Caso essa condição não seja satisfeita, uma `IllegalArgumentException` é lançada, impedindo que o cálculo seja realizado com uma quantidade insuficiente de dados.

Após a validação, o método `calculateVolatility` é utilizado para calcular a volatilidade do ativo. Com 21 preços de fechamento consecutivos, são obtidas 20 variações percentuais entre preços consecutivos. Para cada par de preços, o retorno percentual é calculado pela diferença entre o preço atual e o preço anterior, dividida pelo preço anterior e multiplicada por 100.

Depois de calcular os retornos, o método obtém a média dessas variações. Em seguida, calcula a soma dos quadrados das diferenças entre cada retorno e a média. A variância é obtida dividindo esse valor pelo número de retornos menos um, caracterizando o cálculo da variância amostral. Por fim, a raiz quadrada da variância fornece o desvio padrão dos retornos, utilizado como medida de volatilidade.

O valor da volatilidade é convertido para `BigDecimal` e, por fim, o método `analyze` constrói um evento `AnalyticsUpdated`. Esse evento recebe um novo identificador, mantém o identificador do evento histórico que originou o cálculo por meio do `sourceEventId`, informa o símbolo analisado, define o indicador como `VOLATILITY20D`, armazena o valor da volatilidade calculada e registra o instante em que a análise foi realizada.

4.4 AnalyticsEventProducer.java

Publica o evento no Kafka pelo tópico `analytics.updated`.

5 Integrações Externas

O Analytics Service não realiza chamadas HTTP diretamente para provedores externos de dados de mercado.

Essa responsabilidade pertence ao Market Data Service, que consulta as fontes externas e publica os dados necessários no Kafka.

Consequentemente, o Analytics Service não precisa conhecer detalhes relacionados ao Finnhub ou ao Alpha Vantage. Para realizar seus cálculos, ele depende apenas do contrato dos eventos recebidos.

As principais integrações do Analytics Service são, portanto, com a infraestrutura interna do PulseDesk:

- Apache Kafka, utilizado para receber e publicar eventos;
- Apache Avro, utilizado na definição dos contratos desses eventos;
- Schema Registry, utilizado para armazenar e controlar os schemas dos eventos;
- módulo de contratos compartilhados do PulseDesk, responsável por disponibilizar as classes Java geradas a partir dos schemas Avro.

Essa separação evita que a lógica analítica fique acoplada às APIs responsáveis pela obtenção dos preços.

Caso o Market Data Service passe a utilizar outro provedor no futuro, o Analytics Service pode continuar funcionando sem alterações, desde que o contrato do evento recebido seja mantido compatível.

6 Comunicação por Eventos

O Kafka é o principal mecanismo de comunicação utilizado pelo Analytics Service.

Diferentemente de uma chamada HTTP tradicional, não existe uma requisição direta do Market Data Service para o Analytics Service.

O Market Data Service publica um acontecimento e o Analytics Service reage a ele de forma independente.

6.1 Evento consumido

Na implementação atual, o Analytics Service consome o tópico:

```
historical-market-data.updated
```

As mensagens desse tópico possuem como valor um evento `HistoricalMarketDataUpdated`.

Esse evento fornece ao Analytics Service os dados necessários para o cálculo, incluindo o identificador do evento original, o símbolo do ativo e uma sequência de preços históricos de fechamento.



O método responsável pelo consumo está associado ao consumer group:

```
analytics-service
```

O consumer group permite que diferentes instâncias do Analytics Service trabalhem como um mesmo grupo de consumidores. Caso o serviço seja escalado horizontalmente, as partições do tópico podem ser distribuídas entre essas instâncias.

6.2 Evento produzido

Depois do processamento, o Analytics Service produz um novo evento e o publica no tópico:

```
analytics.updated
```

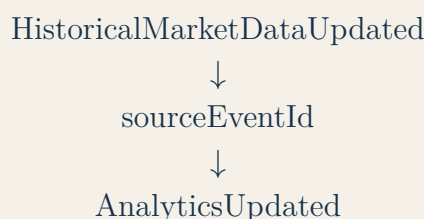
O evento publicado é AnalyticsUpdated.

Ele contém informações como:

- identificador próprio do evento;
- identificador do evento que originou a análise;
- símbolo do ativo;
- indicador calculado;
- valor resultante;
- instante em que a análise foi realizada.

O campo sourceEventId é particularmente importante para a rastreabilidade.

Ele permite estabelecer a relação:



Assim, é possível identificar qual evento de dados históricos originou determinada análise.

7 Contratos de Eventos

A comunicação entre o Market Data Service e o Analytics Service não depende apenas do nome dos tópicos Kafka.

Também é necessário definir a estrutura dos dados enviados nas mensagens.

No PulseDesk, essa estrutura é definida utilizando Apache Avro.



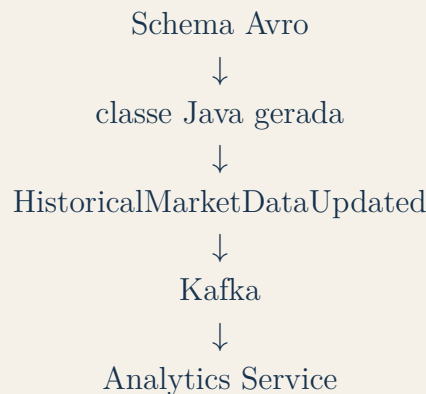
7.1 HistoricalMarketDataUpdated

HistoricalMarketDataUpdated representa o contrato de entrada utilizado no fluxo atual do Analytics Service.

O Market Data Service constrói esse evento e publica uma sequência de preços históricos no tópico historical-market-data.updated.

O Analytics Service importa a classe correspondente através do módulo de contratos compartilhados e recebe o evento diretamente em seu consumer.

Conceitualmente:



O Analytics Service não precisa criar uma estrutura própria para interpretar a mensagem. Producer e consumer trabalham a partir do mesmo contrato.

7.2 AnalyticsUpdated

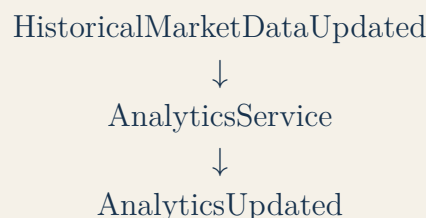
AnalyticsUpdated representa o contrato utilizado para publicar o resultado da análise.

O AnalyticsService utiliza o padrão Builder disponibilizado pela classe gerada para preencher os campos do evento.

Depois da construção, esse objeto é entregue ao AnalyticsEventProducer e serializado para envio ao Kafka.

O evento identifica tanto o resultado analítico quanto sua origem.

Dessa forma, o fluxo completo dos contratos pode ser representado como:



7.3 Schema Registry

O Schema Registry armazena os schemas utilizados na serialização e desserialização dos eventos.

Quando um producer envia um evento Avro, a mensagem pode ser associada ao schema correspondente.

Quando o consumer recebe essa mensagem, o schema permite reconstruir os dados no formato esperado pela aplicação.

Além de armazenar os schemas, o Schema Registry permite controlar sua evolução através de regras de compatibilidade.

Isso é importante em uma arquitetura de microsserviços porque producers e consumers podem ser atualizados de maneira independente.

8 Configuração

8.1 application.yml

O application.yml concentra as configurações externas necessárias para a inicialização do Analytics Service.

Como o fluxo atual do microserviço depende principalmente do Kafka, grande parte dessas propriedades está relacionada à configuração do producer e do consumer.

Entre as informações necessárias estão o endereço utilizado para localizar o broker Kafka e as configurações de serialização e desserialização dos eventos.

O consumer precisa ser capaz de transformar os dados recebidos do Kafka novamente em objetos compatíveis com os contratos Avro utilizados pelo PulseDesk.

O producer realiza o processo inverso: recebe um AnalyticsUpdated criado pelo código Java e utiliza a configuração de serialização para transformá-lo na representação utilizada durante o envio ao Kafka.

A configuração também permite indicar o endereço do Schema Registry, utilizado em conjunto com os serializers e deserializers Avro.

Dentro da rede Docker, o Analytics Service pode acessar Kafka e Schema Registry utilizando os nomes dos respectivos serviços definidos no Compose. Dessa forma, a comunicação entre containers não depende dos endereços utilizados pela máquina host.

O application.yml permite, portanto, manter configurações dependentes do ambiente separadas da lógica Java do serviço.

9 Dependências

9.1 pom.xml

O pom.xml é utilizado pelo Maven para definir as dependências, propriedades e etapas de build do Analytics Service.

Entre as dependências mais importantes para esse microserviço estão Spring for Apache Kafka, Apache Avro e as bibliotecas utilizadas para integração com o Schema Registry.

Spring for Apache Kafka fornece a infraestrutura utilizada tanto pelo consumer quanto pelo producer.

No lado do consumo, essa integração permite utilizar `KafkaListener` para associar métodos Java a tópicos Kafka. Z No lado da produção, fornece o `KafkaTemplate` utilizado para publicar eventos.

Apache Avro é utilizado para representar os contratos dos eventos trocados entre os microsserviços.

As classes `HistoricalMarketDataUpdated` e `AnalyticsUpdated` utilizadas pelo `Analytics Service` correspondem a contratos compartilhados e não a DTOs definidos localmente pelo microsserviço.

A integração com o `Schema Registry` permite serializar e desserializar os eventos de acordo com seus respectivos schemas e controlar a compatibilidade dos contratos.

O `Analytics Service` também utiliza as dependências básicas necessárias para executar uma aplicação `Spring Boot`.

Na implementação atual, o serviço não utiliza `Spring Data JPA`, `PostgreSQL Driver` ou `Flyway`, pois não possui persistência própria em banco de dados.

Também não necessita de uma integração `HTTP` com providers externos para realizar o cálculo atual.

Além das dependências, o `pom.xml` define propriedades do projeto, versão do Java e plugins utilizados pelo Maven para compilar e empacotar a aplicação.

10 Docker

O `Dockerfile` define como o `Analytics Service` é transformado em uma imagem Docker executável.

Durante o processo de build, o código Java é compilado e a aplicação `Spring Boot` é empacotada para que possa ser executada de forma independente dentro de um container.

Quando o container do `Analytics Service` é iniciado, a JVM executa `AnalyticsServiceApplication`.

A partir desse ponto, o `Spring Boot` cria o `ApplicationContext`, identifica os componentes da aplicação e inicializa os beans necessários.

Entre esses componentes estão:

- `HistoricalMarketDataEventConsumer`;
- `AnalyticsService`;
- `AnalyticsEventProducer`.

Depois que o consumer Kafka é inicializado, o serviço passa a aguardar eventos no tópico `historical-market-data.updated`.

Diferentemente de microsserviços que precisam receber chamadas `HTTP` diretamente da máquina host, o `Analytics Service` atual trabalha essencialmente através da rede interna utilizada pelos serviços do `PulseDesk`.

Sua comunicação principal ocorre com Kafka e `Schema Registry`.



No ambiente completo do PulseDesk, o Compose é responsável por executar o Analytics Service junto aos demais componentes e fornecer as configurações necessárias para que ele alcance os serviços de infraestrutura.

A separação entre Dockerfile e Compose permanece:

- Dockerfile: descreve como construir a imagem do Analytics Service;
- compose.yaml: descreve como executar o Analytics Service em conjunto com Kafka, Schema Registry e os demais componentes do PulseDesk.